

rgcam

May 22, 2025

1 rgcam: An R Package for Extracting and Importing GCAM Data

1.1 Maridee Weber

2 Overview

The `rgcam` package provides functions for extracting GCAM data from GCAM output databases and importing it into R for analysis. This notebook demonstrates the primary `rgcam` features; more information can be found on the [rgcam GitHub page](#).

3 Getting started

To use `rgcam` you need a GCAM output database and an XML file containing the queries you want to extract from the database. These queries are modified versions of the queries used in Model Interface. - After running GCAM, your GCAM database can be found in *your_GCAM_Workspace/output* - Model Interface queries can be found in *your_GCAM_Workspace/output/queries/Main_queries.xml*

3.1 Modifying Model Interface queries

To modify a model interface query to work with `rgcam`, you: 1) Copy + paste the query you want from *Main_queries.xml* into a blank XML 2) Add regions to extract data for 3) Add `<aQuery>` before, and `</aQuery>` after 4) Add `<queries>` at the beginning of your XML, and `</queries>` at the end of your XML 5) Save your XML

3.1.1 Example

Here is the query for “population by region” in *Main_queries.xml*

```
<demographicsQuery title="population by region">
  <axis1 name="region">region</axis1>
  <axis2 name="Year">populationMiniCAM</axis2>
  <xPath buildList="true" dataName="total-population" group="false" sumAll="false">demographics/populationMiniCAM/total-population/node()</XPath>
  <comments/>
</demographicsQuery>
```

The modified version for `rgcam` would look like this, where GCAM region “Africa_Southern” has been added to extract data for, and `<aQuery>` and `<queries>` tags have been added.

```

<queries>
  <aQuery>
    <region name="Africa_Southern"/>
    <demographicsQuery title="population by region">
      <axis1 name="region">region</axis1>
      <axis2 name="Year">populationMiniCAM</axis2>
      <xPath buildList="true" dataName="total-population" group="false" sumAll="false">demographics/populationMiniCAM/total-population/node()</XPath>
    <comments/>
    </demographicsQuery>
  </aQuery>
</queries>

```

Your `rgcam` query XML can contain more than one query, too. Having multiple queries would look like this, where the “elec gen by subsector” query has been added for “all-regions”. This means that `rgcam` will extract the data from this query for each GCAM region. Note that the individual queries begin and end with the `<aQuery>` tags, but that the `<queries>` tag is only needed at the beginning and end of the XML.

```

<queries>
  <aQuery>
    <region name="Africa_Southern"/>
    <demographicsQuery title="population by region">
      <axis1 name="region">region</axis1>
      <axis2 name="Year">populationMiniCAM</axis2>
      <xPath buildList="true" dataName="total-population" group="false" sumAll="false">demographics/populationMiniCAM/total-population/node()</XPath>
    <comments/>
    </demographicsQuery>
  </aQuery>
  <aQuery>
    <all-regions/>
    <supplyDemandQuery title="elec gen by subsector">
      <axis1 name="subsector">subsector</axis1>
      <axis2 name="Year">physical-output[@vintage]</axis2>
      <xPath buildList="true" dataName="output" group="false" sumAll="false">*[@type='sector' (:collapse:) and (@name='electricity' or
      @name='base load generation' or @name='intermediate generation' or @name='subpeak generation' or @name='peak generation' or @name='elect_td_bld')/
      *[@type='subsector' and not (@name='elect_td_bld')]/*[@type='output' (:collapse:)]/
      physical-output/node()</XPath>
    <comments/>
    </supplyDemandQuery>
  </aQuery>
</queries>

```

Finally, each of your queries can contain multiple regions, like this. If GCAM-USA was run, US states can be listed to extract data for the individual states.

```

<queries>
  <aQuery>
    <region name="AK"/>
    <region name="KS"/>
    <region name="MD"/>
    <region name="Indonesia"/>
    <region name="EU-12"/>
    <demographicsQuery title="population by region">
      <axis1 name="region">region</axis1>
      <axis2 name="Year">populationMiniCAM</axis2>
      <xPath buildList="true" dataName="total-population" group="false" sumAll="false">demographics/populationMiniCAM/total-population/node()</XPath>
    <comments/>
    </demographicsQuery>
  </aQuery>
</queries>

```

Once your query file is made, you can use `rgcam`.

4 Using `rgcam`

4.1 Connecting your database

The first step to using `rgcam` is to connect it to you GCAM database.

```
[1]: library(rgcam)

host <- "localhost"
conn <- remoteDBConn('gcamv71_training_basexdb', 'training', 'training', host)

# For databases that are on your local machine, you can use conn <-
  ↪ localDBConn("path_to_database", "name_of_database")
# To connect multiple databases, you can use
# conn1 <- localDBConn("path_to_database1", "name_of_database1")
# conn2 <- localDBConn("path_to_database2", "name_of_database2")
```

Database scenarios: GCAM, GCAM_SSP3

The `remoteDBConn` function connects your remote database to R, and running this line tells you which scenario(s) are in your database. In our database, called “gcamv71_training_basexdb”, we have two scenarios: **GCAM**, and **GCAM_SSP3**

4.2 Adding your database to a project file

Once your database is connected to R, you can use the `addScenario` function to extract the data from your scenario(s) and create a data file to save them. Depending on the number of queries and regions data is being extracted for, this step can take some time.

```
[2]: # addScenario(database object, `name of file to store rgcam data`, `path to/
  ↪ name of query XML`, `name of scenario`)
# There are two scenarios, so you use this function twice.
prj <- addScenario(conn, "rgcam_training", queryFile = "/data/queries.xml",
  ↪ scenario = "GCAM", clobber = TRUE)
prj <- addScenario(conn, "rgcam_training", queryFile = "/data/queries.xml",
  ↪ scenario = "GCAM_SSP3", clobber = TRUE)

# For scenarios in different databases, you can use
# prj <- addScenario(conn1, "rgcam_training", queryFile = "/data/queries.xml",
  ↪ scenario = "Scenario1", clobber = TRUE)
# prj <- addScenario(conn2, "rgcam_training", queryFile = "/data/queries.xml",
  ↪ scenario = "Scenario2", clobber = TRUE)
```

Scenario GCAM does not exist in this project. Creating.

Scenario GCAM_SSP3 does not exist in this project. Creating.

After connecting your database and using the `addScenario` function to save GCAM output data to a project file for the first time, you can load your project in `rgcam` the next time using the `loadProject` function. This saves time, as it simply loads the data that has been queried previously without having to reconnect your database and add your scenarios.

```
[3]: prj <- loadProject('rgcam_training')
```

4.3 Retrieving your queries

To retrieve your queries to be able to view the data in R, you can use the `getQuery` function. You assign each query to a dataframe, and use the title of the query from your `rgcam` query XML.

```
[4]: # getQuery(project object, `query title from XML`)  
population <- getQuery(prj, "population by region")  
electricity_generation <- getQuery(prj, "elec gen by subsector")
```

You are now able to view your GCAM output and use it for analysis and visualization through `rchart`, `rmap`, `ggplot2`, or another tool of your choosing.

```
[5]: View(population)
```

Units <chr>	scenario <chr>	region <chr>	year <dbl>	value <dbl>
thous	GCAM	Africa_Eastern	1975	91890
thous	GCAM	Africa_Eastern	1990	145593
thous	GCAM	Africa_Eastern	2005	222829
thous	GCAM	Africa_Eastern	2010	255333
thous	GCAM	Africa_Eastern	2015	292560
thous	GCAM	Africa_Eastern	2020	327652
thous	GCAM	Africa_Eastern	2025	363312
thous	GCAM	Africa_Eastern	2030	398928
thous	GCAM	Africa_Eastern	2035	433912
thous	GCAM	Africa_Eastern	2040	467666
thous	GCAM	Africa_Eastern	2045	499664
thous	GCAM	Africa_Eastern	2050	529402
thous	GCAM	Africa_Eastern	2055	556634
thous	GCAM	Africa_Eastern	2060	581206
thous	GCAM	Africa_Eastern	2065	603136
thous	GCAM	Africa_Eastern	2070	622472
thous	GCAM	Africa_Eastern	2075	639201
thous	GCAM	Africa_Eastern	2080	653078
thous	GCAM	Africa_Eastern	2085	664120
thous	GCAM	Africa_Eastern	2090	672314
thous	GCAM	Africa_Eastern	2095	678024
thous	GCAM	Africa_Eastern	2100	681323
thous	GCAM	Africa_Northern	1975	81330
thous	GCAM	Africa_Northern	1990	119597
thous	GCAM	Africa_Northern	2005	155472
thous	GCAM	Africa_Northern	2010	168395
thous	GCAM	Africa_Northern	2015	184959
thous	GCAM	Africa_Northern	2020	196801
thous	GCAM	Africa_Northern	2025	207480
thous	GCAM	Africa_Northern	2030	216852

A tibble: 1408 × 5

thous	GCAM_SSP3	Taiwan	2065	23557
thous	GCAM_SSP3	Taiwan	2070	23557
thous	GCAM_SSP3	Taiwan	2075	23557
thous	GCAM_SSP3	Taiwan	2080	23557
thous	GCAM_SSP3	Taiwan	2085	23557
thous	GCAM_SSP3	Taiwan	2090	23557
thous	GCAM_SSP3	Taiwan	2095	23557
thous	GCAM_SSP3	Taiwan	2100	23557
thous	GCAM_SSP3	USA	1975	222013
thous	GCAM_SSP3	USA	1990	255627
thous	GCAM_SSP3	USA	2005	298733
thous	GCAM_SSP3	USA	2010	312697
thous	GCAM_SSP3	USA	2015	324365
thous	GCAM_SSP3	USA	2020	332572
thous	GCAM_SSP3	USA	2025	338076
thous	GCAM_SSP3	USA	2030	341189
thous	GCAM_SSP3	USA	2035	342734
thous	GCAM_SSP3	USA	2040	342736
thous	GCAM_SSP3	USA	2045	341067
thous	GCAM_SSP3	USA	2050	337988

```
[6]: View(electricity_generation)
```

Units <chr>	scenario <chr>	region <chr>	subsector <chr>	year <dbl>	value <dbl>
EJ	GCAM	Africa_Eastern	biomass	1990	0.00129301
EJ	GCAM	Africa_Eastern	biomass	2005	0.00210229
EJ	GCAM	Africa_Eastern	biomass	2010	0.00273200
EJ	GCAM	Africa_Eastern	biomass	2015	0.00243691
EJ	GCAM	Africa_Eastern	biomass	2020	0.00435397
EJ	GCAM	Africa_Eastern	biomass	2025	0.01075513
EJ	GCAM	Africa_Eastern	biomass	2030	0.02786302
EJ	GCAM	Africa_Eastern	biomass	2035	0.05595530
EJ	GCAM	Africa_Eastern	biomass	2040	0.10227130
EJ	GCAM	Africa_Eastern	biomass	2045	0.17146150
EJ	GCAM	Africa_Eastern	biomass	2050	0.26383360
EJ	GCAM	Africa_Eastern	biomass	2055	0.38389800
EJ	GCAM	Africa_Eastern	biomass	2060	0.53969700
EJ	GCAM	Africa_Eastern	biomass	2065	0.73580500
EJ	GCAM	Africa_Eastern	biomass	2070	0.96305600
EJ	GCAM	Africa_Eastern	biomass	2075	1.21594400
EJ	GCAM	Africa_Eastern	biomass	2080	1.48130100
EJ	GCAM	Africa_Eastern	biomass	2085	1.75360300
EJ	GCAM	Africa_Eastern	biomass	2090	2.02261000
EJ	GCAM	Africa_Eastern	biomass	2095	2.26164500
EJ	GCAM	Africa_Eastern	biomass	2100	2.51092700
EJ	GCAM	Africa_Eastern	coal	1990	0.00066216
EJ	GCAM	Africa_Eastern	coal	2005	0.00468203
EJ	GCAM	Africa_Eastern	coal	2010	0.00715087
EJ	GCAM	Africa_Eastern	coal	2015	0.00432448
EJ	GCAM	Africa_Eastern	coal	2020	0.00831261
EJ	GCAM	Africa_Eastern	coal	2025	0.01441348
EJ	GCAM	Africa_Eastern	coal	2030	0.02560374
EJ	GCAM	Africa_Eastern	coal	2035	0.04077424
EJ	GCAM	Africa_Eastern	coal	2040	0.06330540

A tibble: 12610 × 6

EJ	GCAM_SSP3	USA	solar	2060	1.3131028
EJ	GCAM_SSP3	USA	solar	2065	1.3768909
EJ	GCAM_SSP3	USA	solar	2070	1.3121513
EJ	GCAM_SSP3	USA	solar	2075	1.2109170
EJ	GCAM_SSP3	USA	solar	2080	1.0246936
EJ	GCAM_SSP3	USA	solar	2085	0.9399493
EJ	GCAM_SSP3	USA	solar	2090	0.9087041
EJ	GCAM_SSP3	USA	solar	2095	0.8475886
EJ	GCAM_SSP3	USA	solar	2100	0.8449479
EJ	GCAM_SSP3	USA	wind	1990	0.0110375
EJ	GCAM_SSP3	USA	wind	2005	0.0643686
EJ	GCAM_SSP3	USA	wind	2010	0.3425130
EJ	GCAM_SSP3	USA	wind	2015	0.6944490
EJ	GCAM_SSP3	USA	wind	2020	1.2186039
EJ	GCAM_SSP3	USA	wind	2025	1.5112684
EJ	GCAM_SSP3	USA	wind	2030	1.8082302
EJ	GCAM_SSP3	USA	wind	2035	2.1042320
EJ	GCAM_SSP3	USA	wind	2040	2.4314445
EJ	GCAM_SSP3	USA	wind	2045	2.2198435
EJ	GCAM_SSP3	USA	wind	2050	2.2149520

```
[7]: # plotting example
library(ggplot2)
library(dplyr)

electricity_generation %>%
  filter(region == "Canada", year > 2010) %>%
  ggplot(aes(x = year, y = value, fill = subsector)) +
  geom_bar(stat = "identity") +
  facet_wrap(~scenario) +
  labs(title = "Electricity Generation in Canada", x = "Year", y = "EJ") +
  theme_bw() +
  theme(legend.title = element_blank())

options(repr.plot.width = 10, repr.plot.height = 4, repr.plot.res = 100)
```

Attaching package: ‘dplyr’

The following objects are masked from ‘package:stats’:

filter, lag

The following objects are masked from ‘package:base’:

intersect, setdiff, setequal, union

Electricity Generation in Canada

